

6CS005

High Performance Computing

Tasks Explanation

Full Name	: Narayan Lohani
Student Number	: 2408869
Course	: BSc (Hons) Computer Science
University Email	: n.lohani@wlv.ac.uk
Tutor	: Bijaya Ghimire
Module Leader	: Yamu Poudel
Submission Date	: January 30, 2026

Table of Contents

Task 1: Word Occurrence Count	1
Problem Statement	1
Solution Overview	1
Key Features	1
Compile and Run	1
How It Works (Gist)	1
Detailed Explanation	2
Error Handling and Robustness	4
Sample Output	5
Files	5
Task 2: Matrix Operations	6
Problem Statement	6
Solution Overview	6
Key Features	6
Compile and Run	6
How It Works (Gist)	6
Detailed Explanation	7
Error Handling and Robustness	9
Sample Output	10
Files	11
Task 3: Brute-Force Password Cracker	12
Problem Statement	12
Solution Overview	12
Key Features	12
Compile and Run	12
How It Works (Gist)	12
Detailed Explanation	13
Error Handling and Robustness	16
Sample Output	17
Files	18

Task 4: Sobel Edge Detection	19
Problem Statement	19
Solution Overview.....	19
Key Features.....	19
Compile and Run.....	19
How It Works (Gist)	19
Detailed Explanation	20
Error Handling and Robustness.....	22
Sample Output.....	23
Files	24

Task 1: Word Occurrence Count

Problem Statement

Given a large text file, efficiently count the frequency of each unique word (case-insensitive, alphanumeric only) using multithreading. The results should be written to a file in sorted order.

Solution Overview

The solution uses POSIX threads (pthreads) to parallelize the word counting process. The input file is read into memory, and the workload is divided among multiple threads. Each thread processes a unique chunk of the file, with boundaries adjusted to ensure no words are split between threads. A global word count array, protected by a mutex, is used to aggregate results from all threads.

Key Features

- **Multithreading:** Utilizes pthreads to process the file in parallel, improving performance on large datasets.
- **Word Boundary Handling:** Thread boundaries are carefully adjusted so that no word is split between two threads, ensuring accurate counting.
- **Case-Insensitive & Alphanumeric:** Words are normalized to lowercase, and only alphanumeric characters are considered.
- **Thread-Safe Aggregation:** A mutex ensures that updates to the shared word count array are safe from race conditions.
- **Sorted Output:** The final word counts are sorted alphabetically before being written to the output file.

Compile and Run

Example:

```
gcc -pthread Main.c -o exe  
./exe WordOccurrenceDataset.txt 8
```

How It Works (Gist)

1. **File Reading:** The entire input file is read into a buffer in memory.
2. **Thread Partitioning:** The buffer is divided into nearly equal-sized chunks, one chunk per thread. Each chunk's start is then adjusted to nearest forward word boundaries to avoid splitting words.
3. **Parallel Processing:** Each thread scans its assigned chunk, extracts words converts them to lowercase, and updates the global word count array.

4. **Synchronization:** A mutex is used to synchronize access to the global word count array, preventing race conditions.
5. **Sorting and Output:** After all threads finish, the word counts are sorted alphabetically and written to result.txt.

Detailed Explanation

1. Initialization and Input Validation

The program begins by validating command-line arguments which contains filename and number of threads. It performs two validations, is the number of arguments three, is the number of threads (third value of the argument) a natural number. If any of the conditions are not met, user-friendly error message is shown.

This program contains three global variables:

- **WordCount words[MAX_WORDS]** : stores words and their respective occurrence count
- **int word_count = 0** : total number of unique words found
- **pthread_mutex_t lock** : Mutex for synchronizing access to shared resources

2. File Reading and Buffer Allocation

With the help of two functions `get_file_size()` and `read_file_to_buffer()`, the entire data of the file is copied into buffer memory.

The function `get_file_size()` uses:

- **fseek** with `SEEK_END` to get to the end of the file
- **ftell** to tell the size of the file (it will be used to allocate memory for buffer)
- **rewind** to get back to the top of the file

The `read_file_to_buffer()` function then utilizes the size provided by `get_file_size()` function to allocate file size + 1 bytes of memory (+1 for null termination) and reads the entire file content into buffer memory using `fread()` file-handling function.

In this way, the data of file is transferred into buffer with memory optimization (exactly required size) and time saving (contents read only once). If at any point, file open is failed or memory allocation is failed or file contents could not be read, then NULL is returned and user is given a friendly error message.

3. Workload Division for threads

As the program now has entire data to process, the data must be divided into nearly equally chunks among the given number of threads for efficient processing. However, the division of data strictly by byte count may occur in the middle of word, this would lead to unintended word split and unexpected outcomes. To overcome this, the thread boundaries are adjusted as follows:

- **First Thread:** Starts at exact byte 0 (beginning of file).

- **Intermediate Threads:** Start boundaries move forward until a non-alphanumeric character (string terminator or new line).
- **Last Thread:** Ends at exact file end.

This ensures while boundaries are adjusted, the initial equal workload division is relatively balanced.

4. Parallel Word Extraction and Update Word Count

Now, that we have threads and their adjusted boundaries over buffer, threads are launched. Each thread will execute **thread_routine_find_word_count()** function, with parameters start index, end index and pointer to the buffer.

The **thread_routine_find_word_count()** function iterates byte-by-byte from its start index to its end index. As it scans, it uses the **isalnum()** function to distinguish between actual word characters and delimiters like newline (`/n`) or terminator character (`/0`). When a valid character is found, it is converted to lowercase using **tolower()** and stored in a local character array until a non-alphanumeric character is encountered, which marks the end of a word.

At this point, the thread terminates the string with a null character and passes it to the **handle_words()** function to be recorded. Because multiple threads are trying to update the same global words array simultaneously, race condition may arise. To prevent that, the **handle_words()** function utilizes a `pthread_mutex_lock` to create a critical section. This lock then acts as a gatekeeper where only one thread can enter the loop to search for or add a word at any given time, while others must wait their turn.

To register the occurrence of the word, the thread first searches the given word in the existing list. If the given word already exists in the list, then increase the count by one, release the lock and return immediately. However, if the word does not exist in the list, then the word is copied into the list using `strncpy` with bounds checking to prevent buffer overflows, set the count to 1 and increase the total unique words count (**word_count**). Then only the lock is released, allowing the next waiting thread to safely update the shared data without causing a race condition.

Each thread uses a local buffer to avoid contention during character accumulation, only accessing shared data when a complete word is ready.

5. Post Processing and Output

The main thread waits for all threads to finish processing the given chunk of buffer. When all threads are done, the global array of words is sorted alphabetically using **qsort()** via lexicographic comparison.

Finally, a new file named `result.txt` is created and the global array of word list is written into the file in “word count” format, one entry per line.

Error Handling and Robustness

1. Input and Command-Line Validation

- **Argument Count:** It checks if (`argc != 3`) to ensure the user has provided exactly two arguments (filename and thread count). If not, it prints a usage guide and exits with an error code.
- **Thread Count:** It uses `atoi()` to convert the thread argument and verifies if number of threads is greater than 0. This prevents the program from attempting to initialize or divide work among zero or negative threads, which would cause a crash.

2. File and Resource Accessibility

- **File Presence:** In `read_file_to_buffer()`, the program checks if the file is missing or the path is incorrect, it returns NULL, and main catches this to prevent further execution.
- **Output Safety:** The `write_results()` function checks if file pointer is accessible before attempting to write the final counts, ensuring the program doesn't crash if it lacks write permissions for the directory.

3. Memory Management and Overflow Prevention

- **Heap Allocation Safety:** Inside `read_file_to_buffer()`, the program checks if buffer memory is created. This handles cases where the system might fail to allocate a large block of memory (e.g., if the file is massive). If allocation fails, it closes the file handle and exits cleanly.
- **Buffer Overflow Protection:** In the thread routine, a check if (`char_index < MAX_WORD_LEN - 1`) is implemented. This prevents a single excessively long "word" in the text file from overflowing the local word array.
- **Resource Cleanup:** The program explicitly frees buffer memory and destroys the mutex lock at the end of main to prevent memory leaks and ensure the operating system reclaims all resources.

4. Thread Safety and Data Integrity

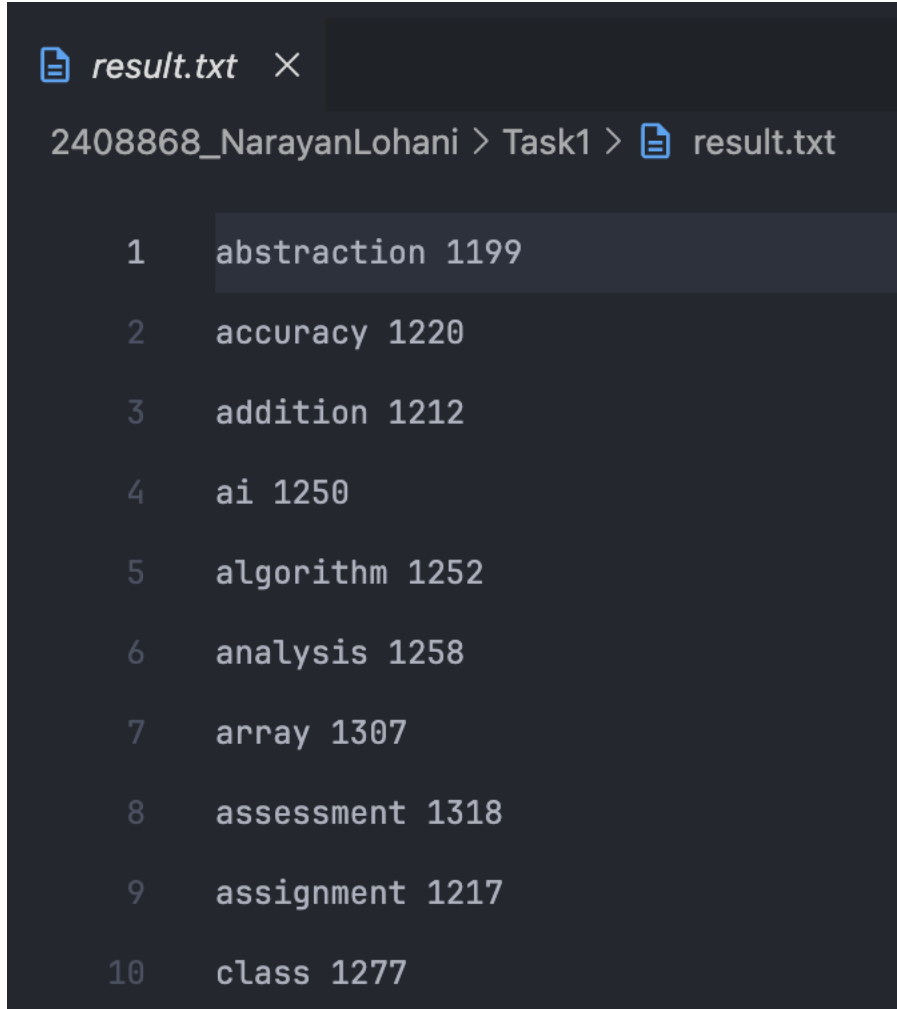
- **Critical Section Protection:** The `handle_words()` function uses `pthread_mutex_lock`. This prevents Race Conditions: a scenario where two threads try to update the `word_count` or the same `WordCount` index simultaneously, which would lead to incorrect counts or memory corruption.

5. Boundary and Edge Case Logic

- **Empty File Handling:** The `get_file_size()` and boundary logic can handle a file size of 0 without crashing.

- **Word Splitting:** The `fix_thread_boundaries()` function ensures no word is chopped in half between two threads. It moves the thread indices until they hit a non-alphanumeric character, ensuring the integrity of the word-counting logic across thread transitions.
- **Trailing Words:** A final `if (char_index > 0)` check exists outside the main thread loop to ensure that if a file ends abruptly without a trailing space, the very last word is still processed and counted.

Sample Output



```
result.txt ×  
2408868_NarayanLohani > Task1 > result.txt  
  
1  abstraction 1199  
2  accuracy 1220  
3  addition 1212  
4  ai 1250  
5  algorithm 1252  
6  analysis 1258  
7  array 1307  
8  assessment 1318  
9  assignment 1217  
10 class 1277
```

Files

- **Main.c: (Created)** Source Code for the multithreaded word counter.
- **WordOccurrenceDataset.txt: (Given)** Input file.
- **result.txt: (Generated)** Output file with sorted word frequencies.

Task 2: Matrix Operations

Problem Statement

Given a file containing multiple matrices, perform a series of mathematical operations on each pair of matrices using parallel processing. The operations include addition, subtraction, element-wise multiplication and division, transpose, and matrix multiplication. The results should be written to a new output file.

Solution Overview

This solution uses **OpenMP** to parallelize matrix operations for improved performance. The program reads matrices from two files (Matrices.txt and MatData.txt), processes them in pairs, and writes the results to corresponding output file named result_Matrices.txt and result_MatData.txt.

Key Features

- **Parallel Processing:** Uses OpenMP to accelerate all matrix operations by distributing work across multiple threads.
- **Robust Input Parsing:** Skips blank lines, validates matrix headers, checks for non-numeric tokens, and ensures correct row/column counts.
- **Dynamic Output Naming:** Output files are automatically named based on the input file for easy identification.
- **Comprehensive Operations:** Supports addition, subtraction, element-wise multiplication or division, transpose, and matrix multiplication based on compatibility of the matrix.
- **Clear Error Reporting:** Provides informative error messages for all input and runtime errors.
- **Memory Safety:** Ensures all allocated memory is properly freed, even in error cases, to prevent memory leaks.

Compile and Run

Example:

```
gcc -fopenmp Main.c -o exe
```

```
./exe MatData.txt 8
```

```
./exe Matrices.txt 12
```

How It Works (Gist)

1. **Input Reading:** The program reads matrices from a file. Each matrix starts with a header line (e.g., 3,4 for a 3x4 matrix), followed by the specified number of data rows.
2. **Matrix Pair Processing:** Matrices are processed in pairs. The program evaluates dimension compatibility for each specific operation before execution.

3. **Parallelism:** All operations are parallelized using OpenMP, with the number of threads specified by the user at runtime.
4. **Output:** Results are written to a file named `result_<inputfilename>`, with clear section headers and CSV-formatted matrices.

Detailed Explanation

1. Initialization and Dynamic Output Naming

The program starts by validating command-line arguments to ensure both a filename and a thread count are provided. It converts the thread argument into an integer and verifies it is a positive natural number. To account for the number of matrix pair operated on, a variable `matrix_pair_count` is initialized to 0.

The output name for file is obtained dynamically using input filename. Instead of using a hardcoded output name, it uses **snprintf()** to create a result filename prefixed with “result_” followed by the original input filename (e.g., if the input is `data.txt`, the output becomes `result_data.txt`). This allows users to process multiple data sets without overwriting previous results.

2. Main Function

To perform operations between every pair of matrices, an infinite loop is created in main function which is broken when EOF (end of file) is reached. The workflow of this infinite loop is as follows:

1. Read a matrix say `matrix_a` using `read_matrix_from_file()` function
2. Read another function say `matrix_b` using same function
3. Increase `matrix_pair_count`
4. Call function `process_matrix_pair()` to operate on of the pair of matrices
5. Free `matrix_a` and `matrix_b`

The loop is broken when there is no more matrix to read from meaning the end of the file. After the loop, there exists an error handling to check if loop was broken by error instead. Then files are closed and program is terminated.

3. Reading matrices from file

The file contains matrices data into two parts: header and matrix values.

The header contains two integers’ rows, columns. In their next rows number of lines, there are columns number of data thus the content of matrices. The two integers of headers are parsed using `sscanf()`. The obtained values are dimension of the matrix which is used to allocate memory for 2-dimensional array of double (`double**`) inside **allocate_matrix()** function. When memory is allocated, the program enters a nested loop to populate the matrix. For each expected row, it reads

the data and uses **strtod()** (**string-to-double**) function to convert comma-separated values into floating-point numbers.

Throughout this whole process, strict error checking is applied. A macro `errno` from `errno.h` is used to detect overflows or non-numeric characters ensuring that each row contains the exact number of columns specified in the header. It also checks for trailing non-numeric characters data the end of the line. If at any point the file ends abruptly or a data mismatch occurs, the function provides a descriptive error message to `stderr`, cleans up any partially allocated memory to prevent leaks and return `NULL`. If everything run successfully, then it updates the output row and column points and returns a pointer to the populated matrix.

This process is repeated twice as matrix are operated in pairs.

4. Parallely Performing Matrix Operations

Now that we have two matrices, **process_matrix_pair()** function is called, which is just a wrapper function to call different other operating function based on the compatibility of two matrices.

For element-wise operations, the dimension of both matrix needs to be same. If the dimension differs, then element-wise operations between pair of matrices is avoided and error message is written in the file. If both the threads have same dimension then, element-wise operation function is called. Before calling the operation functions, if the user-given number of threads is greater than dimension of the matrix, then the number of threads is capped to the row of the matrix. Following functions are called to perform element-wise operations between pair of matrices.

- **add_matrices()**
- **subtract_matrices()**
- **multiply_elementwise()**
- **divide_elementwise()**

All the above functions do follow tasks:

1. Allocate memory of given rows and columns for result matrix using **allocate_matrix()**
2. Marks **#pragma omp parallel for num_threads()**
3. Performs operations (+, -, * /) between elements of [i,j] of both matrix
4. Store the respective resultant element in the respective place
5. Return the pointer of resultant matrix

For division, if the denominator is 0, then nan is written instead.

As transpose of matrix is possible for any dimension of matrix, **transpose_matrix()** is called for both matrices. The number of threads given by user if is greater than row of the matrix, then again it is capped to row of the matrix. The **transpose_matrix()** then calls **allocate_matrix()** with given row and column and using **#pragma omp parallel for num_threads()** shifts element from given

matrix row to resultant matrix columns, interchanging the position of element between row and columns and return the pointer of the resultant pointer.

Finally, matrix multiplication is done if and only if the column of the first matrix is equal to the row of the second matrix, which is the condition to perform matrix multiplication. If the matrices are compatible for multiplication, then threads are again capped to the rows if user given threads are greater than number of rows and **multiply_matrices()** function is called. The **multiply_matrices()** function first calls **allocate_matrix()** to create memory for storing results of matrix multiplication. Then inside **#pragma omp parallel for num_threads()** nested for loop is implemented to calculate the matrix multiplication and it also returns the pointer to the resultant matrix.

5. Post-Processing and Output Formatting

The infinite loop terminates when all the matrices have been processed. For each pair of matrices, **print_matrix()** is called, which writes the result of the matrix operation in the file. First the header is written, which contains the dimensions of both the matrices and that matrix pair number. Then for each, operation if compatible then the resultant matrix dimension is written followed by the content of the matrices. If the operation is not compatible, then clear error with the reason is written in the file. All the output messages are written in the file whether resultant of the matrix or error message.

Error Handling and Robustness

1. Command-Line & Input Validation:

- **Argument Count:** The program checks argc to ensure the user provides both the input file and the thread count; otherwise, it displays a usage guide.
- **Thread Count:** It validates that the number of threads is a positive integer. If the input is zero or negative, it prevents execution and closes any open file pointers.

2. File I/O Safety:

- **Inaccessible Files:** The program verifies the existence and readability of the input file using fopen. If the file cannot be opened, it reports the specific filename and terminates.
- **Output Protection:** It checks if the system can open the result file for writing (e.g., checking for disk permissions or read-only directories).

3. Comprehensive Matrix Parsing:

- **Header Validation:** It ensures the matrix header follows the rows, cols syntax. It specifically checks that dimensions are positive, preventing memory allocation for zero-sized or “negative” matrices.
- **Data Integrity:** During row parsing, the code uses **strtod** combined with **errno** to detect non-numeric characters, overflows, or missing values.
- **Structural Strictness:** It validates that each row contains the exact number of columns specified. If non-numeric data or extra tokens exist at the end of a line, or if the file ends before all rows are read, the program identifies the specific row/column error and stops.

4. Mathematical Safety:

- **Dimension Compatibility:** Before performing addition, subtraction, or multiplication, the program compares row and column counts. If incompatible (e.g., adding a [2,2] to a [3,3], it logs a specific error to the result file rather than crashing.
- **Division by Zero:** In `divide_elementwise`, the program checks if the divisor is 0.0. If so, it manually sets the result to 0.0 / 0.0 (NaN), preventing runtime arithmetic exceptions.

5. Memory Management & Cleanup:

- **Allocation Tracking:** If a matrix is successfully allocated but the file read fails halfway through, the program calls `free_matrix` on the partial data before exiting to ensure no memory leaks occur.
- **Resource Finalization:** Regardless of whether the loop finishes naturally or encounters an EOF, all dynamic buffers are freed, and all file streams are closed using `fclose`.

Sample Output

```
result_MatData.txt X
2408868_NarayanLohani > Task2 > result_MatData.txt

1  =====
2  -----      First: (3, 4)  MATRIX PAIR - 1  Second: (3,4)  -----
3  =====
4
5  Addition (3 x 4)
6  903710.476323, 1008456.397449, 1301741.130630, 344575.269830
7  980489.233941, 642184.151049, 1218029.777154, 1319312.798389
8  882116.994696, 1431965.282759, 1433481.668147, 776431.887252
9
10 Subtraction (3 x 4)
11 -711535.555473, 378723.889685, -557316.945936, -38856.812490
12 262209.847295, 6933.747073, -449269.261636, 670789.294261
13 -353473.282246, 243725.088597, 262291.865645, -676854.689106
14
15 Element-wise Multiply (3 x 4)
16 77602444578.418213, 218388130234.419861, 345981948236.610107, 29305566175.383572
17 223151283463.935425, 103088101752.513138, 320438417145.716553, 322656995674.460938
18 163296757767.435638, 497780663053.861816, 496518167532.492798, 36178551344.230629
19
20 Element-wise Divide (3 x 4)
21 0.118976, 2.202809, 0.400431, 0.797321
22 1.730105, 1.021830, 0.461081, 3.068666
23 0.427847, 1.410229, 1.447907, 0.068519
```

```
result_Matrices.txt X
2408868_NarayanLohani > Task2 > result_Matrices.txt

1 =====
2 ----- First: (10, 10) MATRIX PAIR - 1 Second: (10,10) -----
3 =====
4
5 Addition (10 x 10)
6 1496178.667574, 600109.418153, 762497.164539, 942660.443919, 850609.084965, 1259627.512350, 1364123.331993, 966210.272841, 1015981.518343, 1270261.160219
7 1295604.641482, 1498581.905958, 945456.097576, 1361180.297885, 1405733.045127, 1084650.243464, 1727709.138704, 1042548.365897, 704659.609992, 956337.457952
8 133063.759701, 861736.985551, 1352134.026890, 1552065.214358, 1018605.755191, 894039.501919, 840984.673695, 1411996.287517, 410976.581555, 891274.163563
9 1147353.475614, 1255361.920753, 1383236.003963, 999275.922776, 1340956.934013, 1498282.756776, 1250806.076287, 1043520.543140, 1367154.513639, 962101.983069
10 882796.320151, 1454807.456105, 615256.106737, 1090373.927349, 1371314.044882, 1648376.400062, 1623018.238797, 1789442.634232, 754530.417938, 734424.780105
11 861823.748250, 321746.460627, 984839.378330, 1670549.133251, 1236188.105308, 1355284.664758, 853442.874010, 1216831.008660, 1601483.442344, 289727.342197
12 718059.294752, 609739.573061, 887567.852912, 412572.345435, 1804143.613569, 454671.864076, 1209094.661786, 614524.643884, 1301136.078402, 671455.035323
13 877926.908524, 749999.155252, 1187366.176898, 1145489.687580, 1276873.278473, 1124831.386975, 1803783.514964, 897283.488335, 758685.460047, 1220259.190275
14 962674.454154, 1908688.884595, 1211505.528357, 434320.063074, 627627.972321, 1046238.786781, 752251.032845, 1084500.325718, 754902.222699, 535230.350614
15 582372.936696, 1535916.000089, 955757.601302, 1668900.744376, 1142116.843248, 1765643.377164, 545277.189903, 953056.768956, 873265.118409, 1198423.168222
16
17 Subtraction (10 x 10)
18 100973.686439, 70110.647002, -283973.605124, -582159.722789, -698404.212375, 310117.381504, 375332.547422, 873174.882569, 334431.885185, 401634.269564
19 698494.816487, 195937.602971, 343862.876331, -412417.719680, -447195.049053, -212754.898952, 266147.752849, 343885.416618, -348832.487068, 298664.235138
20 -11095.576676, -181278.369452, 292636.427599, 211349.012649, 203463.989129, -137504.893750, -53193.125678, -460237.234168, -108463.398073, -522819.245691
21 240614.152156, 479344.437368, 411841.327308, -365903.481401, 602605.480727, -347449.415498, 347436.188239, -799780.355723, 608920.230700, -554720.990528
22 723375.150338, 41343.995940, 32328.462459, -362528.176171, -514659.306271, -53512.837361, 176021.860299, -79835.243067, -335787.445006, -69177.433474
23 220409.142719, -140276.196095, 663823.924347, -234512.274473, 219798.428917, 299623.800986, -109506.523254, -725177.883570, 203503.156437, 169218.300905
24 449673.039219, -55038.311287, 101643.855888, -215484.754466, -139555.006599, 150252.017917, -102765.344651, 317097.250457, 47802.651964, -266947.634600
25 -118653.944844, 246450.900736, -94108.878225, 701507.038583, 632921.004450, -28852.165833, 33067.510043, -307773.135106, 63629.342850, -428844.337294
26 564826.755389, -47451.779064, -50939.394027, -91906.290484, -362151.792945, 313925.735157, -440526.743968, 676457.567997, 266691.666430, -403172.319308
27 -356029.733226, 330859.376648, -242768.332238, 267546.911888, -189597.563037, -121503.246898, -415019.918150, 258000.651759, -343266.347258, -151952.398558
28
29 Element-wise Multiply (10 x 10)
30 557088729987.283936, 88803952733.356644, 125190229380.646057, 137424692422.793442, 58941842890.327148, 372622169889.581970, 429989485933.411072, 42781978948.534241,
31 297674094592.902527, 551839046151.541748, 193911388681.399078, 4206808856961.283081, 444025495566.439148, 282800375904.598145, 728536060403.600464, 242162478868.01965,
```

Files

- **Main.c: (Created)** Source code for the parallel matrix operations.
- **MatData.txt / Matrices.txt: (Given)** Input files containing matrix data.
- **result_MatData.txt / result_Matrices.txt: (Generated)** Output file generated by the program.

Task 3: Brute-Force Password Cracker

Problem Statement

Given a file containing SHA512 encrypted passwords, crack its password using brute-force approach by comparing all possible password combinations with target hashes in the encrypted file.

Solution Overview

The solution uses CUDA to parallelize the generation of all possible 4-character password combinations (format: [a-z][a-z][0-9][0-9]). Each thread generates a candidate password and transforms it using a custom cudaCrypt algorithm into 10-character password. The transformed passwords are then encrypted using SHA512 in CPU and compared against target SHA512 hashes to find matches.

Key Features

- **Parallel Password Generation:** Uses CUDA kernel threads to generate all 67,600 possible password combinations in parallel.
- **Custom Transformation Algorithm:** Implements cudaCrypt function that converts 4-character passwords to 10-character passwords,
- **SHA512 Hashing:** Uses the crypt() function with \$6\$AS\$ salt for secure password comparison
- **Progress Tracking:** Displays progress every 12,000 combinations tested
- **Ordered Output:** Results are written in the same order as the input hashes

Compile and Run

Example of creating passwords to crack:

```
gcc CreateEncryptedPasswordText.c -o create -lcrypt
./create
```

Example of cracking passwords:

```
nvcc CrackPassword.cu -o cracker -lcrypt
./cracker EncryptedPasswords.txt
```

How It Works (Gist)

1. Load target SHA512 hashes from the input file
2. Allocate GPU memory and launch CUDA kernel to generate all possible raw passwords, and convert them into 10-character password
3. Copy generated raw passwords back to host memory

4. For each password, compute SHA512 hash and compare against target hashes
5. Write cracked passwords to `result.txt` in the original hash order

Detailed Explanation

For this task, there are two files used.

CreateEncryptedPasswordText.c: To generate 10 SHA encrypted Passwords

CrackPassword.cu: To crack the passwords using brute force comparison.

There are three versions of password involved, namely:

- Raw password: 4-character password, format of [a-z][a-z][0-9][0-9]
- Long password: 10-character password, encrypted version of Raw password
- Hash password: SHA-512 hashed password, encrypted version of Long password

CreateEncryptedPasswordText.c

1. Program Entry & Setup

The program starts by setting random seed of the program. This part is essential to make sure different passwords are generated for every execution of the program. Then two files are opened for writing purpose:

EncryptedPasswords.txt: To store the final hashes

PasswordHint.txt: To store original password (used later to compare against result)

If the file fails to open, the program prints an error and exits immediately. Then two 2D arrays of sizes 10 items of 5 bytes to store raw password (+1 for null terminator) and 10 items of 11 bytes to store long password (+1 for null terminator) are declared.

2. The Processing Loop (Repeated 10 Times)

The program enters a for loop that runs exactly 10 times. For every iteration, it performs the following four sub-steps:

A. Random Generation

It randomly picks four characters based on the ASCII table:

- Two letters between a and z.
- Two digits between 0 and 9.
- It adds a null terminator (\0) to make it a valid string (e.g., "ac42").

B. The cudaCrypt Expansion

The 4-character string is then sent to the cudaCrypt function. This function then:

- Expands: Performs addition/subtraction on the ASCII values of the 4 characters to create 10 characters.
- Bounds Checks: If a calculation results in a character outside 'a'-'z' or '0'-'9', it wraps it back into the valid range.
- Returns: The now 10-character long password is sent back to the loop.

C. Cryptographic Hashing

The 10-character “raw” password is passed to the crypt() function along with the SHA-512 salt (\$6\$AS\$). The library processes the string and returns a long, secure hash string.

D. Temporary Storage & Logging

The original password and the transformed password are saved into the two arrays. The final hash is immediately written to EncryptedPasswords.txt. The conversion raw and long passwords are printed to the terminal to see the program execution.

3. Finalizing the Password Hint File

Once the loop finishes all 10 passwords, the program shifts on PasswordHint.txt where it first writes a header for raw password as “Original Passwords”. It loops through the array of raw password and prints all 10 original 4-char passwords. Then it writes a second header as “cudaCrypt Hashes”. It loops through the array of long password and prints all 10 expanded 10-char passwords.

4. Cleanup & Exit

The program calls fclose() on both file pointers to ensure all data is physically saved from the buffer to the hard drive. Then a final “success” message is printed to the console.

CrackPassword.cu

1. Initialization & Data Loading

The program starts by checking the command-line arguments for provided filename. If filename is not provided, then correct usage to run the file is printed. It opens the text file and reads up to 10 SHA-512 hashes into the targetHashes array. If file does not exist or could not read from the file, then user-friendly error message is printed. Then the file is read and cleans up the strings by removing trailing newlines (\n) obtained from file.

Then memory is allocated for the device to contain all the possible combinations of raw password. As there are 67,600 possible combinations (two alphabets and two number: $26*26*10*10 = 67,600$) 67,600 bytes of memory is allocated on the GPU using **cudaMalloc()** function.

2. Parallel Generation (The GPU Phase)

To achieve maximum parallelism, each thread works on a single combination of password. So, for 67,600 passwords at least 67,600 threads are required. The dimension of block (number of threads)

is set to (16,16) which creates 256 threads in 2D. Then, the number of blocks is calculated dynamically by dividing total combination (67,600) by threads per block (256) which results in 265 blocks in 1D. Then the kernel function **generatePasswordsKernel()** is launched on the GPU with following configuration:

gridDim(265) and blockDim(16,16).

And now Brute Force logic begins. All threads of the GPU should create a unique raw password. To do this, each threads has its unique ID calculated using:

idx = blockIdx.x * blockDim.x * blockDim.y + threadIdx.y * blockDim.x + threadIdx.x

for 1D grid and 2D block. This unique thread index is converted into 4-character raw password.

For example: Index 0 is aa00, and index 67599 is zz99. It is done using following formula:

Letter 1	'a' + idx / (26 * 10 * 10)
Letter 2	'a' + (idx % (26 * 10 * 10)) / (10 * 10)
Digit 1	(idx % 100) / 10
Digit 2	idx % 10

The accumulation of above four characters forms unique raw password combination for each thread of GPU. However, as we launched kernel with 265 blocks and 256 threads, we have 67,840 threads which is more than required. If the idx is greater than 67,600 then that threads simply do nothing. This ensures no extra resources is wasted.

Now that each thread has a raw password it needs to be converted into long password, for which **cudaCryptDevice()** function is called. The **cudaCryptDevice()** uses the same logic as in **CreateEncryptedPasswordText.c** to convert raw password to long password. The CPU waits until the GPU has finished generating every single possibility using **cudaDeviceSynchronize()**.

3. Data Transfer

The CPU allocates a block of RAM using **malloc()** to receive the data in CPU. Then using **cudaMemcpy()** with fourth parameter as “**cudaMemcpyDeviceToHost**” all the generated 67,600 passwords are transferred from the GPU memory to CPU memory.

4. Brute Force Password Comparison (The CPU Phase)

The program now iterates through the 67,600 possibilities on the CPU to find matches:

- For each long password, it calls the **crypt()** function from **crypt.h** header file using the fixed SALT (\$\$AS\$).
- It compares the resulting hash against the 10 target hashes loaded at the start.

- If a hash matches:
 - It marks that hash as cracked so it doesn't waste time looking for it again.
 - It converts the loop index back into the original raw password format using same logic as GPU's conversion from unique index to raw password
 - It prints the success message to the console.

If all 10 hashes are found before reaching the end of the list, the loop terminates early to save time.

5. Output & Cleanup

The program opens result.txt for writing mode and writes the successfully cracked 4-character passwords (raw passwords) in the order they were provided. It then frees the system RAM using free() and the GPU memory using cudaFree(). To avoid the usage, explicitly cudaDeviceReset() is called which shuts down the CUDA context and the program terminates.

Error Handling and Robustness

1. Argument Validation

The program requires a source of target hashes to function. Without a file path, the program has nothing to crack. So, at the very start of main, the program checks if (argc != 2). If the user fails to provide a filename, it prints a "Usage" guide and terminates with exit code 1.

2. File I/O Validation

The files might be missing, locked by another process, or have restricted permissions. Before reading hashes, it checks if file pointer is not NULL. If the input file cannot be opened, it alerts the user and exits. Another file-handling is also required when writing result.txt. The logic of writing in file is wrapped inside NULL check for output file pointer to ensure the program doesn't attempt to write to a null pointer if the disk is full or the directory is read-only.

3. Newline Handling and Buffer Safety

The fgets function includes the newline character (\n) in the string. If left intact, the strcmp function would never find a match because the hash would look like hash\n instead of hash. The program checks the end of each loaded hash and manually replaces the \n with a null terminator \0. It also limits the read to MAX_HASH_LENGTH to prevent buffer overflows.

4. CUDA Error Checking & Device Safety

GPU memory is finite, and kernels can fail due to configuration errors or hardware timeouts.

- **Memory Allocation:** It captures the cudaError_t during cudaMalloc. If the GPU is out of memory, it prints the specific CUDA error string and exits.
- **Kernel Execution:** It uses cudaGetLastError() after the kernel finishes. This catches silent failures (like invalid indexing) that might have occurred during the parallel execution.

- **Synchronization:** `cudaDeviceSynchronize()` ensures the CPU doesn't try to read data before the GPU has finished writing it.

5. Algorithmic Boundary Management (`cudaCryptDevice`)

The mathematical transformation (e.g., +2, -3) could push a character code outside the printable ASCII range for letters and numbers. Inside the `__device__` function, a loop checks every character. If a character exceeds the ASCII value for 'z' or falls below 'a', it uses a wrapping formula to force the character back into the valid [a-z] or [0-9] in ASCII range.

6. Progress Indicators and Logic Guards

Brute-forcing 67,600 combinations in serial computing takes time. Without feedback, the status of program if it is running successfully or if it is frozen cannot be understood. That's why for every 12,000 combinations; the CPU prints a progress report to the console.

The loop uses the condition `crackedCount < numHashes`. Once all targets are found, the program stops immediately rather than finishing all 67,600 checks, significantly saving time.

Sample Output

```
EncryptedPasswords.txt ×
2408868_NarayanLohani > Task3 > EncryptedPasswords.txt

1  $6$AS$xiC.a0dsY0PdobEKT3qMbteboZu0QLj0R2Tpc/.i4Cb3C73w2eYESEuN72onhERv52pjwikaui7onF.Jm61a.
2  $6$AS$Gfk.194ZmE88xUjIUuSiFx0ZRRQWoASXx/LFM8.TzaF0WHL0ih8EXG6S8z2WX1BYRF7emQS7Lhby9V1bAruVh/
3  $6$AS$rmagLqt867zf0eYP3rqXKvrDCXyg2FvMh.TxPme21UJeJRVIoLPRIIWvc0PhpImqbIsuDRF8JohaXfVykEg17/
4  $6$AS$89t15h.f7.bsSEBFhTBut5X9s6GHL0P6gsAg6pfEXyNvLWUDeS1PkEKgHKky6NRJyMi9wvS.0la12LFouz0Nc.
5  $6$AS$N9V9Klm/zhKrVyZ.j2D.UkaZ3a1dvx2hsbVsT4bZLTtknu3iUafKn2qBXhyNZ4vc13gv9CuYtYy9XDGuMQ3M.
6  $6$AS$21neSv8M5yGfZiy.xNs4rwJ4TK8zWu1geSZQgtFT0EqcFzo.7QA0ZSXNemNkXC5BXDJEDyNgHnp30b5IPkaM.
7  $6$AS$ZHa8IpaHxCeqCx7bo6ZENYC4UE3MpDcWLMWgcCvAx9TC0UQF6wJEYRrE7vv2yETnrNENcpe9i00neMeDAAAU41
8  $6$AS$0iKydy9yxyn/x560IcMyXYVj3LKh8qu7so9.25D030xLy2n7KPsZk63vcLnDzvrwvhtkt733/LaKIh/Spo6Ttp1
9  $6$AS$0Hp8btsRKQWipsyP946ZrHlzyTp1pL6W.C2kycmplREJHk0nR9bMSwoAXzt6G6V5TSLwMKh9c2DFz2KI6ane0
10 $6$AS$Rp.W.5Nds3Tj4MU/x1.1KY66RdSBq8esTrirFhVYLwmMfb1Ez2ccHyfg8ew0MZGRZ35JQaggsuYeHr08ueQE1
11
```

```
result.txt ×
2408868_NarayanLohani > Task3 > result.txt

1  qf50
2  fz74
3  nw48
4  wo24
5  yf02
6  sk25
7  rp92
8  wm51
9  sm39
10 lv54
```

```
PasswordHint.txt ×
2408868_NarayanLohani > Task3 > PasswordHint.txt

1  Original Passwords:
2
3  1. qf50
4  2. fz74
5  3. nw48
6  4. wo24
7  5. yf02
8  6. sk25
9  7. rp92
10 8. wm51
11 9. sm39
12 10. lv54
13
14 cudaCrypt Hashes:
15
16 1. sorice7344
17 2. hdgdwy9580
18 3. ploztv6234
19 4. yuxrln4080
20 5. bwzice2262
21 6. uqtnhj4091
22 7. tpssmo2762
23 8. yuxpjl7353
24 9. uqtpjl5145
25 10. njmysu7380
```

Files

CreateEncryptedPasswordText.c: (Created) Source code to generate 10 random encrypted passwords.

CrackPassword.cu: (Created) CUDA source code to brute force crack passwords.

EncryptedPasswords.txt: (Generated) Stores encrypted passwords (to be cracked).

PasswordHint.txt: (Generated) Stores the raw and cudaCrypt hash of passwords stored in EncryptedPasswords.txt (can be compared with result.txt)

result.txt: (Generated) Stores the cracked passwords from EncryptedPasswords.txt

Task 4: Sobel Edge Detection

Problem Statement

Given a PNG image, detect the edges of the image using Sobel edge detection algorithm powered by GPU acceleration.

Solution Overview

The solution uses CUDA to perform Sobel edge detection on PNG images. The program loads an RGBA image using LodePNG, transfers it to GPU memory, and applies both grayscale conversion and Sobel convolution in a single optimized kernel. The resulting edge map is sent back to host and exported as a new PNG image.

Key Features

- **Single Optimized Kernel:** Performs both grayscale conversion and Sobel filtering are performed in one kernel execution
- **Constant Memory Usage:** Sobel kernels (Gx and Gy) stored in CUDA constant memory for fast access
- **Zero-Padding Boundary Handling:** Handles image borders gracefully using zero-padding
- **Automatic Output Naming:** Auto-generates output filename if not specified
- **2D Thread Configuration:** Uses 16×16 thread blocks for efficient 2D image processing
- **Luminance-based Grayscale:** Uses standard luminance formula ($0.299R + 0.587G + 0.114B$)

Compile and Run

Example:

```
nvcc lodepng.cpp sobel_gpu.cu -o sobel_gpu -lm
```

```
./sobel_gpu car.png # Auto-generates output name
```

```
./sobel_gpu car.png car_output.png # Custom output name
```

How It Works (Gist)

1. Load the input PNG image as 32-bit RGBA using LodePNG
2. Allocate GPU memory and transfer the RGBA image to device
3. Launch CUDA kernel that converts to grayscale and applies Sobel operators
4. Compute gradient magnitude $\sqrt{G_x^2 + G_y^2}$ for each pixel (the edge map)
5. Copy the edge map back to host and convert it to RGBA format and save the PNG image

Detailed Explanation

1. Program Entry & Argument Parsing

The program starts by checking `argc` to ensure an input file is provided. Then the name of the output file is determined. If the output filename is not provided explicitly, `generate_output_filename()` function runs, which strips `.png` from the input file name and appends “`_sobel_edge_output.png`” at the suffix.

2. Image Loading (Host/CPU)

The program calls the `lodepng_decode32_file()` from LodePNG library. It reads the file from the hard drive, decompresses the PNG format, and expands it into a large array of unsigned char in the memory. Each pixel now occupies 4 bytes (Red, Green, Blue, Alpha). The width and height are stored to calculate the total number of pixels.

3. GPU Memory Preparation

The CPU now tells GPU to assign two memory blocks using `cudaMalloc`:

- **d_rgba:** to store original color image.
- **d_output:** to store final black-and-white edge map.

After the memory allocation CPU copies the raw image data from the CPU memory to GPU memory using `cudaMemcpy` with fourth parameter set to “`cudaMemcpyHostToDevice`”.

4. Kernel Configuration (Grid & Block)

- **dim3 blockDim(16, 16):** 2D block is created which store 16 threads in x axis and 16 threads in y axis, resulting in 256 total number of threads.
- **dim3 gridDim(...):** The dimension of grid is calculated dynamically based on blocks required to cover the whole image. For example, for image of 1024 x 1024, it creates a grid of 64 x 64 blocks. If the number of blocks in any axis is not a perfect factor of the number of pixels in that axis, then number of blocks is rounded to upper limit.

5. Parallel Execution (sobel_kernel)

Finally, the CPU launches the kernel and then waits for all threads to finish work using `cudaDeviceSynchronize()`. Inside the GPU, thousands of threads are created to run this logic simultaneously for their assigned pixel `[x, y]`.

For 1D grid and 2D block, x and y are calculated as:

$$x = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

$$y = \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y}$$

this ensures the pixel [x][y] is processed by thread[x][y]. Each thread now performs following task:

- **Iterate Neighborhood:** Each thread looks at its 3 x 3 grid centered on its pixel.
- **Boundary Check:** If a neighbor is outside the image (the edges of the image), it uses 0 (black) as padding for its neighbors' values.
- **Grayscale conversion:** For each of the 9 pixels in the neighborhood, the thread extracts R, G, and B, applies the luminance formula, and gets a single pixel value.
- **Convolution:** It multiplies these values by the d_Gx, and d_Gy masks stored in Constant Memory.

```
__constant__ int d_Gx[9] = { -1, 0, 1, -2, 0, 2, -1, 0, 1 };  
__constant__ int d_Gy[9] = { -1, -2, -1, 0, 0, 0, 1, 2, 1 };
```

The sum of the gx and gy at any given pixel are calculated by multiplying the neighboring pixels of that pixel with the gradient's constants. However, as the image are accessed with 2D indexing but the gradient constant is flat 1D, the 1D index for the constant masks is calculated using the formula $kidx = (ky + 1) * 3 + (kx + 1)$. Because the loops for ky and kx range from -1 to 1, adding 1 shifts these coordinates to a range of 0 to 2, effectively mapping the 3 x 3 spatial neighborhood into a linear offset from 0 to 8. This allows the GPU to efficiently retrieve the corresponding Sobel weights from the flat d_Gx and d_Gy arrays in constant memory, ensuring that the top-left neighbor is multiplied by the first element of the mask, the center pixel by the fifth, and the bottom-right neighbor by the ninth.

- **Magnitude:** The strength of the edge is calculated by performing the square root of the sum of squares.

$$\text{Mathematically, } G = \sqrt{(Gx)^2 + (Gy)^2}$$

- **Output:** The final intensity is written to d_output.

6. Data Retrieval (Device to Host)

Once the GPU is finished, the CPU retrieves the processed edge map (the single-channel data) back from the GPU memory into the CPU memory using cudaMemcpy with fourth parameter this time set to "cudaMemcpyDeviceToHost".

7. Post-Processing & Image Saving

PNG images are usually made up of 4 channels, so the program loops through the grayscale output and assigns the same value to Red, Green, and Blue, and sets Alpha to 255 (255 for maximum visibility). Finally, **lodepng_encode32_file()** function from the LodePNG library is used to compress the raw array back into a .png file and saves it to the hard drive.

8. Cleanup

The memory allocated on GPU is released using **cudaFree()** and memory allocated on CPU is released using **free()**. The program terminates here.

Error Handling and Robustness

The program incorporates several layers of error handling to ensure stability during GPU-accelerated image processing. Below are the specific areas where errors are managed:

Command-Line Argument Flexibility:

The program validates the user's input arguments at the very start. If the required input filename is missing, it provides a usage guide and exits gracefully. Furthermore, it also includes a safety mechanism to automatically generate a valid output filename if the user fails to provide one, ensuring the program always has a valid path to save its results.

CUDA API Validation (CUDA_CHECK Macro):

The program defines a custom `CUDA_CHECK` macro that wraps critical functions like `cudaMalloc`, `cudaMemcpy`, and `cudaDeviceSynchronize`. If a call fails (e.g., due to an incompatible GPU or driver issues), it retrieves the specific error string from the CUDA runtime, prints the exact file and line number to `stderr`, and terminates the program safely to prevent undefined behavior.

Input/Output File Validation:

The program utilizes `LodePNG` error codes to verify the integrity of the image files. It checks if the input PNG can be successfully decoded from the disk and if the final edge map can be encoded back into a file. If a file is corrupted, missing, or has an unsupported format, the program reports the specific error text and halts execution.

Memory Allocation Safeguards:

The code performs dual validation for memory resources:

- **Host (CPU):** It checks if `malloc` successfully allocated the CPU memory for the output buffer, preventing crashes if the system is out of memory.
- **Device (GPU):** It uses the `CUDA_CHECK` macro on `cudaMalloc` to ensure the GPU has enough available memory to store the RGBA image and the resulting edge map.

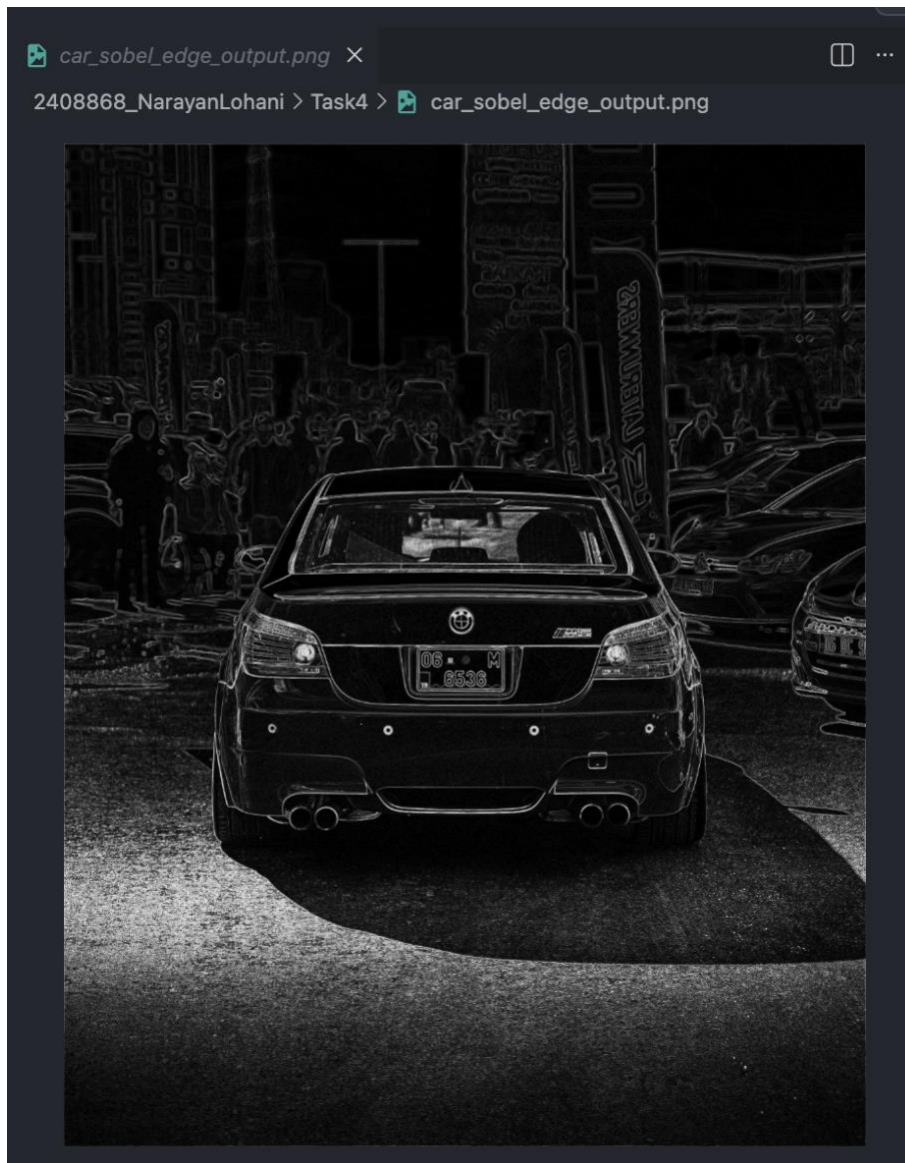
Robust Kernel Execution Checks:

After the kernel launch, the program calls `cudaGetLastError()`. This is crucial for catching asynchronous errors such as invalid memory access within the GPU or trying to launch more threads than the hardware supports that would otherwise go unnoticed until much later in the execution.

Boundary and Overflow Handling:

- **Zero-Padding:** Within the kernel, a logical check ($nx \geq 0 \ \&\& \ nx < \text{width} \dots$) ensures that the code doesn't attempt to read pixels outside the image dimensions. If a neighbor is out of bounds, it defaults to a black pixel (0), preventing memory access violations (segmentation faults on the GPU).
- **Luminance and Magnitude Clamping:** The `clamp255` function ensures that mathematical results exceeding 255 (due to high-intensity gradients) or falling below 0 are capped, preventing integer overflow when storing values back into the unsigned char array.

Sample Output



Files

- **lodepng.cpp: (Given)** Library file for LodePNG function definitions.
- **lodepng.h: (Given)** Header file for LodePNG functions
- **sobel_gup.cu: (Created)** CUDA source code for edge detection using Sobel algorithm
- **car.png: (Picked)** Input image PNG
- **car_sobel_edge_output.png: (Generated)** Edge map of the car.png